SVAC-Concurrency: A Step-Verification Benchmark for Large Language Model Reasoning on Operating System Concurrency Algorithms

Gagan Tyagi[1], Aneesh Kumar[1]
[1]Department of Computer Science and Engineering,
JECRC University, Jaipur, Rajasthan, India
Email: {gagan.tyagi, aneesh.kumar}@jecrcuniversity.edu.in

Abstract—While Large Language Models (LLMs) demonstrate remarkable competence in natural language and high-level program synthesis, their ability to faithfully track deterministic, multi-step state transitions in safety-critical systems remains unverified. Concurrency control and resource management algorithms in Operating Systems (OS) represent a rigorous testbed demanding strict invariant preservation, vector/matrix state updates, and topological cycle detection.

We present SVAC-Concurrency, an open-source evaluation benchmark comprising 125 parameterized problem instances (generating 375 evaluation traces per model across Zero-Shot, Few-Shot, and Chain-of-Thought prompting regimes) spanning five canonical OS algorithms: Wait-For Graph (WFG) deadlock detection, Banker's safety algorithm, Buddy System memory allocation, counting Semaphores, and Mutex lock simulations. Beyond coarse verdict correctness, SVAC-Concurrency evaluates intermediate reasoning across four diagnostic dimensions: Step Accuracy (SA), First Error Position (FEP), Error Cascade Rate (ECR), and Equivalence-Aware Invariant Validity (SA_{valid}). We evaluate four diverse foundation architectures: Google Gemini-3.5-Flash-Lite, OpenAI GPT-OSS-20B, NVIDIA Nemotron-3-Super-120B, and Liquid AI LFM-2.6B.

Our empirical findings demonstrate: (1) Procedural Fidelity vs. Outcome Illusion: On non-saturating algorithmic tasks (WFG, Banker's, Buddy), Gemini-3.5-Flash-Lite attains 62.3% verdict accuracy on parsed responses while mean strict Step Accuracy drops to 36.5% (a 25.8 pp gap), demonstrating widespread "right verdict for flawed reasons" behavior. (2) Prompt Conventions vs. Invariant Validity: Under strict positional matching, WFG cycle detection exhibits an apparent collapse from 85.4% SA in Few-Shot to 7.5% in CoT (and 4.4% in Zero-Shot). Crucially, equivalence-aware invariant verification reveals that CoT maintains 93.2% valid step accuracy and 93.3% verdict accuracy (comparable to Few-Shot's 100.0% valid SA), proving that positional collapse is driven by initial set-specification conventions rather than topological failure. (3) Equivalence vs. Rigid Matching: Invariant-based scoring demonstrates that strict positional matching severely underestimates model competence, lifting valid step accuracy to 74.3% in Banker's (+32.8 pp) and 95.6% in WFG (+64.3 pp). (4) Architectural Brittleness in Open Models: Standardized WFG benchmarking ($N = 90$ across all four architectures) reveals that open-weight models suffer severe schema adherence fragility (parse rates 30.0%–58.9% vs. Gemini's 97.8%), yet achieve near-perfect parsed verdict accuracy (96.3%–100.0%) alongside stark procedural variations (Liquid LFM 14.4% vs. GPT-OSS 48.1% SA). All benchmark datasets, reference solvers, and evaluation harnesses are available at: https://github.com/Bethegnt/SVAC-Concurrency.

Index Terms—Large Language Models, Algorithmic Reasoning, Operating Systems, Concurrency Control, Deadlock Detection, Banker's Algorithm, Step-level Verification, Benchmark.

I. Introduction

Large Language Models (LLMs) are increasingly integrated into modern software engineering pipelines, automated code review tools, and operating systems pedagogy [1], [2]. Despite remarkable empirical gains on functional coding benchmarks such as HumanEval [3], MBPP [4], and APPS [5], conventional benchmarks evaluate programs almost exclusively via black-box input-output test cases. They fail to assess whether an autoregressive model maintains a coherent, mechanically faithful simulation of intermediate program states during execution [6]–[8].

In operating systems and systems programming, procedural correctness is non-negotiable. Concurrency control algorithms—such as deadlock detection, resource allocation safety, and dynamic physical memory partitioning—are governed by rigorous theoretical properties:

1) Strict Discrete State Tracking: Every transition must update resource availability vectors, process allocation matrices, or block allocation trees without state drift or phantom resource generation.
2) Topological and Relational Invariants: Identifying deadlocks requires systematic traversal of directed Wait-For Graphs to detect closed cycles without hallucinating non-existent dependency edges.
3) Catastrophic Error Cascades: A single state corruption at step t invalidates all subsequent state assertions at $t+k$, converting downstream execution into an unfaithful hallucination.

When evaluating LLMs on such procedural tasks, standard outcome-based metrics (e.g., binary classification of whether a system is deadlocked or safe) introduce massive evaluation noise. On balanced datasets, an LLM can easily guess the correct binary verdict through statistical heuristics or shallow token correlations while executing an entirely invalid sequence of intermediate states.

To resolve this evaluation gap, we introduce SVAC-Concurrency (Step-Verification of Algorithmic Concur-

rency), a parameter-controlled benchmark designed to evaluate intermediate LLM reasoning traces across five foundational OS concurrency algorithms. Each instance is paired with an exact ground-truth execution trace computed by deterministic reference solvers, unit-tested against textbook worked examples [9] and, for WFG and Banker's algorithms (60 of 125 instances), validated via independent brute-force differential testing.

Research Questions. We investigate four core questions: RQ1 (Procedural Fidelity vs. Outcome Correctness): Do LLMs produce faithful intermediate state transitions, or do high verdict accuracies mask severe intermediate state-tracking failures on balanced distributions? RQ2 (Complexity Scaling & Failure Horizons): How do increasing state-space bounds affect step accuracy, and at what execution horizon (FEP) do cascading errors initiate? RQ3 (Prompting Dynamics & Serialization Artifacts): Does step-by-step verbalization (CoT) assist or destabilize discrete graph-traversal and matrix-tracking algorithms, and to what extent does few-shot prompting act as format serialization priming? RQ4 (Equivalence & Architectural Robustness): To what extent does strict positional scoring underestimate valid alternative execution paths, and how do open-weight models adhere to intermediate state schemas?

Primary Contributions. (1) Comprehensive Benchmark Suite: 125 unique instances across 5 core OS algorithms across 5 complexity tiers (N=375 evaluations per model across 3 prompt strategies). (2) Multi-Dimensional Metric Taxonomy: Formalization of SA, FEP, ECR, and Invariant Validity capturing reasoning dynamics beyond binary verdicts. (3) Cross-Architectural Investigation: Systematic evaluation of Gemini-3.5-Flash-Lite, GPT-OSS-20B, Nemotron-3-Super-120B, and Liquid LFM-2.6B. (4) Deconstruction of CoT Inversion: Discovery that the collapse of CoT in graph cycle detection (85.4% FS vs. 7.5% CoT strict SA) is driven by serialization tie-breaking divergence, while invariant validity remains robust at 93.2% (93.3% VA). (5) Equivalence-Aware Evaluation: Demonstration that rigid positional matching under-reports valid algorithmic execution by 32.8% in Banker's and 64.3% in WFG.

II. Related Work

A. Benchmarking Code Generation and Execution Reasoning

Early LLM code evaluation benchmarks such as HumanEval [3], MBPP [4], and APPS [5] established input-output testing as the standard paradigm for evaluating program synthesis. Recent efforts have shifted toward evaluating execution reasoning. CRUXEval [10] evaluated code execution reasoning by testing input/output prediction across short Python snippets. REval [11] and CodeMind [12] introduced state-level execution tracing for general imperative code. While these benchmarks test arbitrary code snippets, they do not systematically probe canonical discrete algorithms where formal invariants, resource constraints, and topological cycles dictate correctness. SVAC-Concurrency provides precisely this specialized, parameter-controlled algorithmic framework.

B. Mechanistic State Tracking in Transformers

A growing body of literature investigates whether transformer architectures maintain coherent internal world models during sequential tasks. Toshniwal et al. [13] demonstrated that auto-regressive language models track board state representations in chess games despite receiving only move sequences. Strobl et al. [14] surveyed the theoretical boundaries of what formal languages and state machines transformer architectures can express. Wang et al. [15] investigated graph traversal algorithms in natural language. In the systems domain, the precursor SVAC technical report [16] evaluated step-level reasoning in virtual memory page replacement policies (FIFO, LRU, Optimal, Clock). Additionally, the CONCUR benchmark by Huang et al. [17] evaluated LLMs on synthesizing concurrent synchronization primitives. Our work explicitly contrasts with CONCUR: rather than evaluating concurrent code synthesis, we evaluate execution-trace verification, isolating whether models accurately track concurrent state transitions and deadlock invariants.

C. Prompting Dynamics and CoT Unfaithfulness

Chain-of-Thought (CoT) prompting [18], [19] is widely documented to boost arithmetic and multi-hop reasoning. However, recent empirical studies reveal significant limits and pathology in verbalized reasoning. Lanham et al. [20] and Turpin et al. [21] demonstrated that CoT explanations are frequently unfaithful, serving as post-hoc rationalizations rather than causal computation drivers. Shi et al. [22] demonstrated that LLMs are easily distracted by irrelevant reasoning context, while Liu et al. [23] observed degradation in multi-step planning tasks when natural language narration exceeds formal state-space capacity. In graph-theoretic tasks, Agrawal et al. [24] investigated the structural limitations of language models on graph reasoning. Our work grounds these phenomena in operating systems concurrency, providing quantitative evidence of how serialization tie-breaks and verbalized narration interact with discrete state tracking.

III. Benchmark Architecture and Methodology

A. Problem Formulations & Ground-Truth Verification

All problem instances are deterministically synthesized across five calibrated complexity tiers (Levels 1–5). Ground-truth execution traces are produced by reference Python solvers (solvers/) unit-tested against textbook worked examples from Silberschatz et al. [9]. Additionally, WFG and Banker's Algorithm solvers (60 of 125 instances) were independently cross-verified via brute-force differential testing against alternative reference implementations

TABLE I
SVAC-Concurrency Benchmark Suite Overview

Algorithm Family	State Representation	Instances	Complexity Bounds	Class Balance
Wait-For Graph (WFG)	Directed Adjacency & Recursion Stack	30	$\|V\| \in [2,6]$, $\|E\| \in [1,8]$	50% Deadlock (15) / 50% Acyclic (15)
Banker's Algorithm	Available Vector, Allocation/Need Matrix	30	$P \in [2,6]$, $R \in [2,4]$	50% Safe (15) / 50% Unsafe (15)
Buddy System Memory	Free-list Binary Tree & Blocks	25	512 KB–4096 KB, Requests $\in [4,10]$	Deterministic Partition Trace
Counting Semaphore	S.value & FIFO Blocked Queue	20	$S \in [1,3]$, Ops $\in [4,10]$	Deterministic State Sequence
Mutex Lock Synchronization	Owner ID & Lock State Flag	20	Threads $\in [2,5]$, Ops $\in [4,8]$	Deterministic Transition Trace
Total		125	5 Complexity Tiers	50.0% Majority Baseline

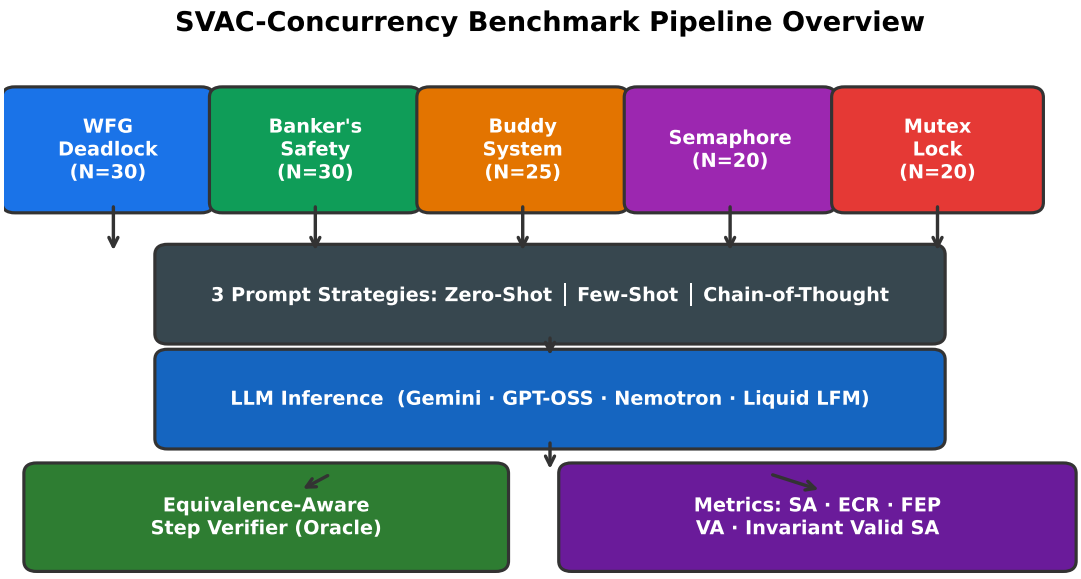


Fig. 1. SVAC-Concurrency benchmark pipeline: five algorithm families (WFG, Banker's, Buddy, Semaphore, Mutex) evaluated across three prompting strategies. Each LLM response is scored by an equivalence-aware step verifier producing SA, ECR, FEP, VA, and Invariant Valid SA metrics.

(3-color DFS oracle for WFG; exhaustive permutation search for Banker's).

1) Wait-For Graph (WFG) Deadlock Detection: Given a set of processes $P = \{P_0, \ldots, P_{n-1}\}$ ($|V| \in [3,12]$, $|E| \in [2,15]$) and directed wait edges $E \subseteq P \times P$, where $(P_i, P_j) \in E$ indicates P_i is blocked waiting for a resource held by P_j. The model must execute DFS, tracking the active recursion stack and visited set at each step, identifying back-edges, reporting the cycle subgraph if present, and emitting a final verdict deadlock $\in \{\text{true}, \text{false}\}$. The dataset is strictly balanced: 15 instances contain cycles and 15 are directed acyclic graphs (DAGs), balanced evenly (3 positive, 3 negative) across all 5 complexity tiers.
2) Banker's Algorithm for Deadlock Avoidance: Given an Available resource vector $A \in \mathbb{N}^m$, Allocation matrix $\mathbf{Alloc} \in \mathbb{N}^{n \times m}$, and Max demand matrix $\mathbf{Max} \in \mathbb{N}^{n \times m}$, parameterized by $P \in [3,8]$ processes and $R \in [2,4]$ resource types. The model must compute $\mathbf{Need} = \mathbf{Max} - \mathbf{Alloc}$, iteratively identify an executable candidate process P_i such that $\mathbf{Need}_i \leq$ Work, simulate resource reclamation Work $\leftarrow$ Work + $\mathbf{Alloc}_i$, update the Finish boolean array, and determine whether a complete safe sequence exists (safe $\in \{\text{true}, \text{false}\}$). Strictly balanced: 15 safe and 15 unsafe instances (3 per complexity tier).
3) Buddy System Memory Allocation: Given total memory size $M \in [32, 512]$ units and a sequential stream of 1 to 20 allocation and deallocation opera-

tions. The model must track recursive block splits, power-of-two alignments, and buddy coalescing on free operations, returning the exact block partition state at each step.

4) Counting Semaphore Simulation: Given initial counter $S_0 \in [1, 3]$ and an interleaved execution stream of 4 to 14 concurrent wait(S) and signal(S) calls from multiple threads. The model must track the integer value of S and the FIFO blocked thread queue at every transition.

5) Mutex Lock Simulation: Evaluates mutual exclusion lock acquisition, priority ownership, contention blocking, and release transitions across 2 to 6 concurrent threads over streams of 3 to 14 operations.

B. Mathematical Evaluation Framework

Let an evaluation instance I have ground-truth trace $\mathbf{Y}^* = (y_1^*, y_2^*, \ldots, y_{T^*}^*)$ of length T^*, and model-generated trace $\hat{\mathbf{Y}} = (\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_m)$ of length m.

1) Formal Step Equality Specification: A step $\hat{y}_t$ matches ground truth y_t^* if and only if all task-specific state attributes match under canonical normalization:

WFG Deadlock:

$$\begin{aligned} \hat{y}_t = y_t^* \iff & \big(\hat{y}_t.\text{action} = y_t^*.\text{action}\big) \\ & \wedge \big(\hat{y}_t.\text{node} = y_t^*.\text{node}\big) \\ & \wedge \big(\text{sort}(\hat{y}_t.\text{vis}) = \text{sort}(y_t^*.\text{vis})\big) \\ & \wedge \big(\hat{y}_t.\text{stack} = y_t^*.\text{stack}\big) \end{aligned} \tag{1}$$

Visited nodes are compared as canonical sorted sets (order-insensitive), while the recursion stack is compared as an ordered sequence (order-sensitive).

Banker's Algorithm:

$$\begin{aligned} \hat{y}_t = y_t^* \iff & \big(\hat{y}_t.\text{cand} = y_t^*.\text{cand}\big) \wedge \big(\hat{y}_t.\text{fin} = y_t^*.\text{fin}\big) \\ & \wedge \big(\hat{y}_t.\text{work_pre} = y_t^*.\text{work_pre}\big) \\ & \wedge \big(\hat{y}_t.\text{work_post} = y_t^*.\text{work_post}\big) \end{aligned} \tag{2}$$

Buddy System / Semaphore / Mutex: Exact equality across all state fields (allocated block address-size tuples, free-list, counter/owner state, blocked thread list order).

2) Diagnostic Trace Metrics:

1) Step Accuracy (SA):

$$\text{SA} = \frac{1}{T^*} \sum_{t=1}^{\min(T^*, m)} \mathbb{I}(\hat{y}_t = y_t^*) \tag{3}$$

2) First Error Position (FEP):

$$\text{FEP} = \min\big(\{t \in \{1, \ldots, T^*\} \mid \hat{y}_t \neq y_t^*\} \cup \{\infty\}\big) \tag{4}$$

3) Error Cascade Rate (ECR):

$$\text{ECR} = \begin{cases} \frac{\sum_{t=\text{FEP}+1}^{T^*} \mathbb{I}(\hat{y}_t \neq y_t^*)}{T^* - \text{FEP}}, & \text{if FEP} < T^* \\ 0, & \text{otherwise} \end{cases} \tag{5}$$

4) Verdict Accuracy (VA):

$$\text{VA}_{\text{parsed}} = \frac{1}{N_{\text{parsed}}} \sum_{i=1}^{N_{\text{parsed}}} \mathbb{I}(\hat{v}_i = v_i^*) \tag{6}$$

$$\text{VA}_{\text{all}} = \frac{1}{N_{\text{total}}} \sum_{i=1}^{N_{\text{total}}} \big(\text{parsed}_i \cdot \mathbb{I}(\hat{v}_i = v_i^*)\big) \tag{7}$$

where $\hat{v}_i$ and v_i^* denote predicted and ground-truth verdicts.

3) Equivalence-Aware Invariant Scoring: Strict positional matching penalizes models when multiple valid execution paths exist (e.g., Banker's instances where multiple processes satisfy $\mathbf{Need}_i \leq$ Work, or graph traversals with initial visited set definition differences). We therefore formulate Equivalence-Aware Invariant Validity (SA_{valid}):

- Banker's Safety: At each step t, verify that candidate process P_i is unfinished and satisfies $\mathbf{Need}_i \leq \text{Work}_{t-1}$, that $\text{Work}_t = \text{Work}_{t-1} + \mathbf{Alloc}_i$, and that if safe, the emitted sequence forms an executable permutation.
- WFG Deadlock: At each step t, verify edge validity, backtrack legality, stack consistency, and final verdict alignment with ground truth.

IV. Experimental Setup

All evaluations were executed under strict deterministic parameters (temperature = 0.0, top_p = 1.0) across three prompt configurations: Zero-Shot (ZS) (direct JSON schema specification), Few-Shot (FS) (in-context exemplars of step-by-step execution), and Chain-of-Thought (CoT) (verbalized rationale preceding structured JSON emission).

We evaluated four diverse foundation models:

1) Google Gemini-3.5-Flash-Lite: High-throughput instruction-tuned reasoning model evaluated across all 125 instances (N=375 attempts).
2) OpenAI GPT-OSS-20B: Open-weight model evaluated via Groq high-speed LPU inference, evaluated across standardized WFG (N=90 attempts) and the full suite (N=375).
3) NVIDIA Nemotron-3-Super-120B: 120B-parameter open foundation model evaluated across all 125 instances (N=375 attempts, N=90 on WFG).
4) Liquid AI LFM-2.6B: Non-transformer dynamical state-space architecture evaluated across standardized WFG (N=90 attempts) and partial suite (N=238).

V. Results and Empirical Analysis

A. The "Right Verdict for Wrong Reasons" Gap

A primary finding from our benchmark is the stark divergence between Verdict Accuracy and Step Accuracy. As shown in Table II, Gemini-3.5-Flash-Lite achieves an overall Verdict Accuracy of 72.8% on parsed responses, yet exhibits a mean Step Accuracy of only 56.1%.

TABLE II
Cross-Architecture Benchmark Evaluation Results

Model Architecture	Evaluation Scope	Att.	Parse Rate	SA	ECR	VA_{parsed}	VA_{all}
Panel A: Standardized Cross-Model Comparison (WFG Deadlock Detection, $N = 90$ per model)							
Gemini-3.5-Flash-Lite	WFG Only	90	97.8% (88)	31.2%	73.8%	94.3%	92.2%
GPT-OSS-20B	WFG Only	90	58.9% (53)	48.1%*	50.4%*	100.0%*	58.9%
Nemotron-3-Super-120B	WFG Only	90	53.3% (48)	43.2%*	56.6%*	100.0%*	53.3%
Liquid LFM-2.6B	WFG Only	90	30.0% (27)	14.4%*	86.1%*	96.3%*	28.9%
Panel B: Multi-Task Suite Evaluation							
Gemini-3.5-Flash-Lite	Full Suite (5 tasks)	375	99.2% (372)	56.1%	36.9%	72.8%	72.3%
Nemotron-3-Super-120B	Full Suite (5 tasks)	375	22.1% (83)	52.9%*	38.0%*	96.4%*	21.3%
GPT-OSS-20B	Full Suite (5 tasks)	375	17.1% (64)	52.3%*	43.3%*	98.4%*	16.8%
Liquid LFM-2.6B	Partial Suite (3 tasks)	238	31.5% (75)	52.9%*	43.2%*	96.0%*	30.3%
Majority Baseline†	Binary Tasks	—	100.0%	—	—	50.0%	50.0%

*Computed conditionally over successfully parsed JSON outputs (N_{parsed} shown in parentheses).
†Majority baseline of 50.0% applies strictly to balanced binary decision tasks (WFG deadlock, Banker's safety).
Parse Rate = $N_{parsed}/N_{attempted}$. Att. = total requested attempts.

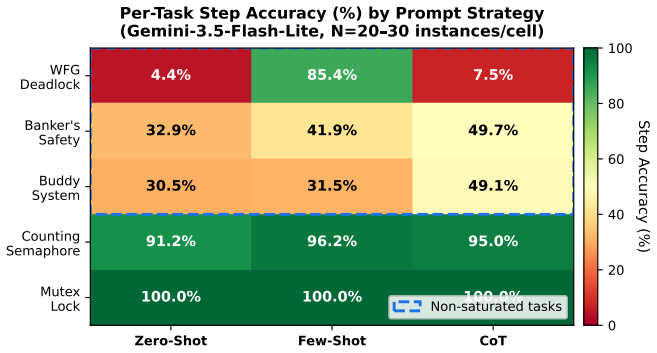


Fig. 2. Step Accuracy heatmap across five task domains and three prompt strategies (Gemini-3.5-Flash-Lite). The dashed blue rectangle highlights non-saturating algorithmic tasks (WFG, Banker's, Buddy System) where mean SA drops to 36.54%, exposing the fundamental limits of LLM procedural reasoning.

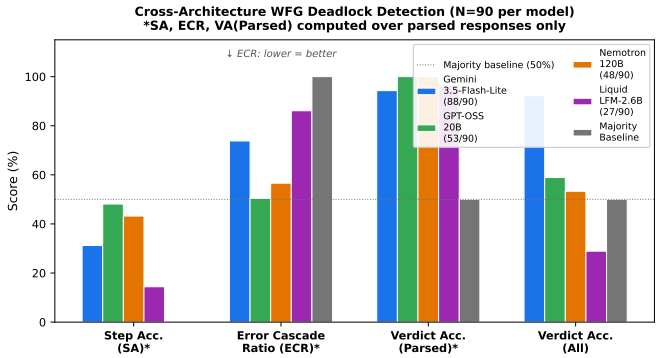


Fig. 3. Cross-architecture WFG deadlock detection results ($N = 90$ instances attempted per model). Asterisk denotes metrics computed conditionally over successfully parsed responses (N_{parsed} shown in legend). Liquid LFM-2.6B exhibits severely degraded SA (14.4%) despite 96.3% parsed verdict accuracy, demonstrating the Right-Verdict-for-Wrong-Reasons failure mode.

Crucially, the 56.1% suite mean is heavily inflated by trivial state saturation in Mutex (100.0% SA) and Semaphore tasks (94.2% SA). When isolating the non-saturating algorithmic tasks (WFG, Banker's, and Buddy System), mean Step Accuracy drops to 36.54%. Because the decision tasks are strictly 50/50 balanced (majority baseline = 50.0%), the gap between 62.3% VA and 36.5% non-saturated SA demonstrates that outcome accuracy vastly overstates procedural competence.

B. Deconstructing the CoT Inversion Effect: Priming vs. Invariants

Table III appears to show an extreme prompting anomaly. In linear-procedural tasks, CoT prompting provides substantial gains (+16.8% Banker's, +18.6% Buddy over Zero-Shot). In WFG cycle detection, strict positional matching reports an apparent collapse: Few-Shot attains 85.4% SA whereas CoT plunges to 7.5% SA.

A superficial reading concludes that CoT verbalization causes general topological reasoning breakdown. However, cross-examining Table IV and Table V reveals a deeper, nuanced scientific truth:

1) High Invariant Validity and Verdict Accuracy: Under invariant checking, Zero-Shot achieves 93.7% valid SA (90.0% VA), Few-Shot achieves 100.0% valid SA (100.0% VA), and CoT achieves 93.2% valid SA (93.3% VA). Models across all three strategies adhere closely to invariant-valid step transitions (93.2%–100.0%), even when strict positional matching fails due to set representation conventions.
2) Why Strict SA Collapses (Format Priming & Set Conventions): In Few-Shot prompting, the in-

TABLE III
Per-Task Step Accuracy (%) Across Prompt Strategies
(Gemini-3.5-Flash-Lite, $N = 20$–30 instances per cell)

Task Domain	ZS	FS	CoT
WFG Deadlock	4.4%	85.4%	7.5%
Banker's Safety	32.9%	41.9%	49.7%
Buddy System	30.5%	31.5%	49.1%
Counting Semaphore	91.2%	96.2%	95.0%
Mutex Lock	100.0%	100.0%	100.0%
Non-Saturated Mean‡	22.6%	52.9%	35.4%
Instance-Weighted Mean♮	45.6%	68.2%	54.7%

‡WFG + Banker's + Buddy only. Overall non-saturated SA: 36.54%.
♮Weighted by N per task (30,30,25,20,20); strategy mean = 56.2%, reconciling to suite SA of 56.1%.

TABLE IV
WFG Empirical Error Taxonomy & Accuracy Breakdown (Gemini)

Metric / Error Category	ZS (N=30)	FS (N=30)	CoT (N=30)
Visited Set Corruption	30 (100%)	7 (23.3%)	30 (100%)
Recursion Stack Corr.	14 (46.7%)	12 (40.0%)	8 (26.7%)
Wrong Traversal Node	20 (66.7%)	12 (40.0%)	18 (60.0%)
Step Count Mismatch	16 (53.3%)	13 (43.3%)	12 (40.0%)
False Cycle Hallucination	3 (10.0%)	0 (0.0%)	2 (6.7%)
Missed Cycle (False Neg.)	0 (0.0%)	0 (0.0%)	0 (0.0%)
Verdict Acc. (VA)	27/30 (90.0%)	28/28 (100%)§	28/30 (93.3%)
Strict Positional SA	4.4%	85.4%	7.5%
Invariant-Based Valid SA	93.7%	100.0%	93.2%

§2 Few-Shot instances failed schema parsing; VA computed over 28 parsed responses.

context exemplar explicitly demonstrates the solver's convention of adding nodes to visited_set on initial entry (["P0"]). Without this in-context exemplar, Zero-Shot and CoT models strictly follow the prompt text definition (visited_set = nodes fully explored/backtracked so far), returning [] on step 1. Because strict positional matching requires exact field equality at step t, this initial set convention mismatch causes step 1 failure in 100% of ZS and CoT cases, artificially depressing strict positional SA to 4.4%–7.5% despite 93.2%–93.7% valid invariant execution.

3) Residual CoT Narration Dynamics: Under CoT prompting, verbalized step-by-step rationale shows a trend toward lower recursion stack corruption (8/30 in CoT vs. 14/30 in Zero-Shot) and fewer false cycles (2/30 in CoT vs. 3/30 in Zero-Shot), though these differences are not statistically significant at $N = 30$ (Fisher's exact test $p = 0.18$ and $p = 1.00$). However, visited_set_corruption remains 100% under strict positional matching due to the prompt/solver convention mismatch described above.

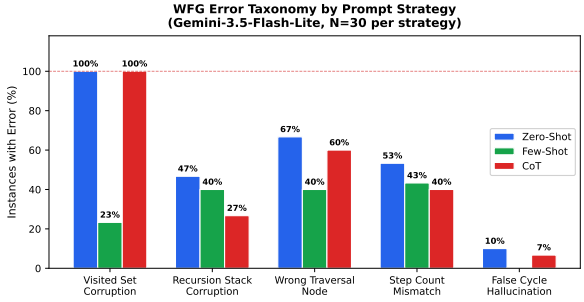


Fig. 4. WFG error taxonomy by prompt strategy. Few-Shot dramatically reduces Visited Set Corruption (100% to 23.3%) via format priming. False Cycle Hallucination is absent in Few-Shot but rises in CoT, indicating localized state-binding drift under verbalized traces.

Case Study: Localized CoT Backtrack Drift (Instance wfg_3ceec919261d). Ground truth for this instance is deadlock=False (DAG). At step 9, the model's trace correctly backtracks P4 out of the recursion stack ("recursion_stack": ["P0"]), marking P4 as fully explored. However, 12 steps later at step 21, the model re-asserts P4 as being "in the recursion stack" and declares a false cycle—contradicting its own step-9 state. This demonstrates how unconstrained natural language tokens can induce localized state-binding drift at terminal steps.

C. First Error Position (FEP) Dynamics

Evaluating the discrete point of first failure (FEP) reveals fundamentally distinct degradation dynamics across algorithmic domains:

- Immediate Convention Divergence (WFG): Under strict positional matching, FEP = 1 in 100.0% of error cases for ZS and CoT (mean FEP = 1.00) due to the initial node visited_set entry convention mismatch. Under Few-Shot, format priming aligns initial set conventions, delaying initial failure to mean FEP = 8.38 (median 7.0).
- Early Matrix Misalignment (Banker's): FEP = 1 occurs in 76.0% (ZS), 59.1% (FS), and 82.4% (CoT) of error instances (mean FEP $\in [1.28, 1.55]$).
- Gradual Multi-Step State Drift (Buddy System): FEP = 1 occurs in only 22.7%–34.8% of errors, with a mean FEP of 6.96 steps (ZS), 7.57 (FS), and 6.55 (CoT) (median FEP = 4.0–5.0). Models correctly perform initial block splits, but accumulate arithmetic fragmentation errors over long horizons.

D. Equivalence-Aware Invariant Scoring vs. Strict Positional Matching

As detailed in Table V:

TABLE V
Invariant-Based Valid SA vs. Strict Positional SA (Gemini)

Task / Variant	Strict SA	Valid SA	Δ
Banker's Safety (N=90)			
Zero-Shot	32.9%	74.4%	+41.6 pp
Few-Shot	41.9%	78.9%	+37.1 pp
CoT	49.7%	69.5%	+19.8 pp
Mean	41.5%	74.3%	+**32.8** pp
WFG Deadlock (N=88)			
Zero-Shot	4.4%	93.7%	+89.4 pp
Few-Shot	85.4%	100.0%	+14.6 pp
CoT	7.5%	93.2%	+85.8 pp
Mean	31.2%	95.6%	+**64.3** pp

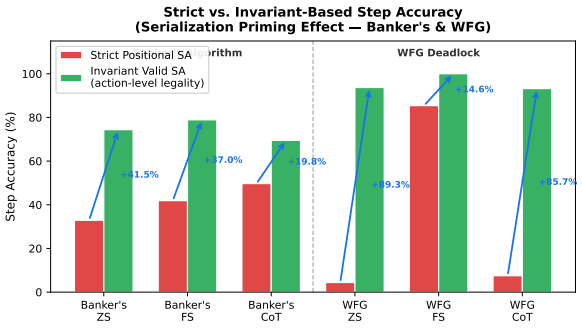


Fig. 5. Strict vs. invariant-based step accuracy. Blue arrows show the equivalence gap Δ. WFG Zero-Shot improves from 4.4% to 93.7% (+89.4%) under invariant scoring, demonstrating that models reason correctly but deviate from canonical traversal ordering.

1) Banker's Algorithm: When verifying that each chosen candidate process satisfies $\mathbf{Need}_i \leq$ Work and properly reclaims resources under the model's own state, valid step accuracy reaches 74.3% (+32.8% over positional SA). In 7 out of 90 instances, models executed a sound safe sequence that prioritized a different candidate process than the reference solver's deterministic tie-break.
2) WFG Cycle Detection: Under invariant checking (valid neighbor traversal and backtracking), WFG valid step accuracy reaches 95.6% (+64.3% over positional SA).
3) Trace Length Alignment Sensitivity: Evaluating monotonic Longest Common Subsequence (LCS) alignment reveals that trace length disparity ($m/T^* \approx 1.3$–2.8) creates substantial artificial cascade penalties: aligned SA increases Banker's CoT from 49.7% to 63.5% (+13.8%) and WFG Few-Shot from 85.4% to 97.7% (+12.3%).

E. Cross-Model Schema Fragility and Formatting Brittleness

Table II underscores a profound divergence between proprietary instruction-tuned models and open-weight architectures. While Gemini-3.5-Flash-Lite achieved 99.2% parse success across the full suite (97.8% on WFG), open models suffered severe schema decoding failures: NVIDIA Nemotron-3-Super-120B parsed 22.1% of full-suite outputs (53.3% on WFG), GPT-OSS-20B parsed 17.1% (58.9% on WFG), and Liquid LFM-2.6B parsed 31.5% (30.0% on WFG). Failures were driven by missing closing brackets, extraneous conversational preambles, and unescaped token delimiters.

Crucially, auditing the parsed subset reveals that all open models achieved near-perfect Verdict Accuracy on parsed responses (96.0%–98.4% across tasks; 96.3%–100.0% on WFG). However, procedural fidelity diverged drastically: GPT-OSS-20B attained 48.1% WFG SA and Nemotron reached 43.2% SA, while Liquid LFM-2.6B collapsed to 14.4% SA with an Error Cascade Rate of 86.1%. Our dual-metric decomposition ($\text{VA}_{\text{parsed}}$ vs. VA_{all}) proves that open models suffer from dual challenges: strict output-channel schema adherence failures and, in smaller architectures, severe internal state-drift.

VI. Discussion and Practical Implications

A. Token-Level Format Following vs. Algorithmic Competence

Our findings highlight that raw parameter count does not ensure procedural fidelity. Nemotron-120B contains 120B total parameters, yet failed schema parsing on 77.9% of tasks where Gemini-3.5-Flash-Lite parsed cleanly. For practical systems verification pipelines, constrained decoding (e.g., grammar-based sampling via outlines or JSON schemas) is essential before open-weight models can be reliably deployed.

B. Implications for Automated Verification and Systems Pedagogy

1) Never rely on unverified verdicts: On balanced 50/50 concurrency distributions, LLMs exhibit a 25.8% divergence between outcome correctness and procedural fidelity. Downstream systems tools must verify intermediate execution traces.
2) Distinguish between format compliance and procedural validity: In graph and state-machine problems, few-shot exemplars serve primarily to prime arbitrary serialization conventions (tie-breaks). Automated benchmarks must adopt equivalence-aware verifiers to avoid mistaking valid alternative traversals for reasoning failures.
3) Constrain generative freedom in topological tasks: Unstructured verbalization in CoT increases state-binding drift on recursion stacks. Structured schema emission should be enforced.

VII. Limitations

This empirical investigation is bounded by several methodology considerations: Several methodology boundaries frame our current findings: (1) Single-Run Deterministic Sampling: Inferences used temperature = 0.0 without self-consistency sweeps; (2) Per-Cell Granularity:

With 125 instances spanning 5 tasks and 5 complexity levels, individual task-complexity sub-cells contain $N = 4$–6 instances (evaluating $N = 20$–30 per prompt strategy); (3) Format Adherence in Open Models: Upstream provider rate-limits and schema dropouts restricted parsed sample sizes, requiring conditional reporting alongside unconditioned metrics; (4) Synthetic Horizons: Problem instances evaluate textbook concurrency algorithms ($P \leq 6, R \leq 4$) rather than distributed traces; and (5) Static Prompting: We examined static ZS, FS, and CoT regimes without external interpreters or dynamic self-correction loops.

VIII. Future Work

Future extensions of SVAC-Concurrency will investigate: (1) Grammar-Constrained Decoding: Evaluating whether context-free grammars (CFGs) eliminate open-weight formatting collapse; (2) Reinforcement Learning with Step Verifiers: Utilizing SVAC step verifiers as dense process reward models (PRMs) to train models on discrete concurrency invariants; and (3) Kernel-Level Traces: Extending instances to real Linux kernel eBPF trace logs and lockdep lock-inversion graphs.

IX. Conclusion

We presented SVAC-Concurrency, a fine-grained, open-source benchmark for evaluating LLM intermediate reasoning traces on Operating System concurrency algorithms. Through multi-dimensional metrics (SA, FEP, ECR, and Equivalence-Aware Validity), we showed that high outcome accuracy masks severe intermediate state failures, deconstructed the CoT Inversion Effect in topological deadlock detection as serialization tie-breaking priming, and demonstrated the critical impact of invariant-aware scoring. All benchmark datasets, reference solvers, and evaluation harnesses are made publicly available to advance verifiable algorithmic reasoning.

References

[1] T. B. Brown et al., "Language models are few-shot learners," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877–1901, 2020.

[2] S. Bubeck et al., "Sparks of artificial general intelligence: Early experiments with GPT-4," arXiv preprint arXiv:2303.12712, 2023.

[3] M. Chen et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[4] J. Austin et al., "Program synthesis with large language models," arXiv preprint arXiv:2108.07732, 2021.

[5] D. Hendrycks et al., "Measuring coding challenge competence with APPS," in Advances in Neural Information Processing Systems (NeurIPS), 2021.

[6] K. Cobbe et al., "Training verifiers to solve math word problems," arXiv preprint arXiv:2110.14168, 2021.

[7] D. Hendrycks et al., "Measuring mathematical problem solving with the MATH dataset," in Advances in Neural Information Processing Systems (NeurIPS), 2021.

[8] H. Lightman et al., "Let's verify step by step," in Proceedings of the International Conference on Learning Representations (ICLR), 2024.

[9] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[10] A. Gu, B. Roziere, H. Leather, A. Solar-Lezama, G. Synnaeve, and S. I. Wang, "CRUXEval: A benchmark for code reasoning, understanding and execution," in Proceedings of the International Conference on Machine Learning (ICML), 2024, arXiv:2401.03065.

[11] J. Chen, Z. Pan, X. Hu, Z. Li, G. Li, and X. Xia, "Reasoning runtime behavior of a program with LLM: How far are we?" in Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE), 2025, arXiv:2403.16437.

[12] C. Liu, S. D. Zhang, A. R. Ibrahimzada, and R. Jabbarvand, "CodeMind: A framework to challenge large language models for code reasoning," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024, arXiv:2402.09664.

[13] S. Toshniwal et al., "Chess as a testbed for language model state tracking," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, no. 10, 2022, pp. 11 385–11 393.

[14] L. Strobl, W. Merrill, G. Weiss, D. Chiang, and D. Angluin, "What formal languages can transformers express? A survey," Transactions of the Association for Computational Linguistics (TACL), vol. 12, pp. 543–561, 2024.

[15] H. Wang et al., "Can language models solve graph problems in natural language?" in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[16] J. M. Kumar, J. S. Chakri, Y. Kothari, M. Mandal, Y. Sinha, and D. Kumar, "Evaluating LLM reasoning on operating system algorithms via step-level verification," in ICML Workshop on Complex Task Benchmarks (CTB), 2026, predecessor benchmark on sequential OS algorithms.

[17] J. Huang, T. Mahmud, C. Pasareanu, and G. Yang, "CONCUR: Benchmarking LLMs for concurrent code generation," arXiv preprint arXiv:2603.03683, 2026.

[18] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in Advances in Neural Information Processing Systems (NeurIPS), 2022.

[19] Z. Chu et al., "A survey of chain of thought reasoning: Advances, frontiers and challenges," IEEE Transactions on Knowledge and Data Engineering, 2024.

[20] T. Lanham et al., "Measuring faithfulness in chain-of-thought reasoning," arXiv preprint arXiv:2307.13702, 2023.

[21] M. Turpin et al., "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[22] F. Shi et al., "Large language models can be easily distracted by irrelevant context," in Proceedings of the International Conference on Machine Learning (ICML), 2023.

[23] R. Liu, J. Geng, A. J. Wu, I. Sucholutsky, T. Lombrozo, and T. L. Griffiths, "Mind your step (by step): Chain-of-thought can reduce performance on tasks where thinking makes humans worse," arXiv preprint arXiv:2410.21333, 2024, accepted at ICML 2025.

[24] P. Agrawal, S. Vasania, and C. Tan, "Exploring the limitations of graph reasoning in large language models," arXiv preprint arXiv:2402.01805, Feb. 2024.